

TEKNİK ENTEGRASYON KILAVUZU

E-İmza API Platformu

Server-side ve client-side imzalama, doğrulama, demo ve offline masaüstü uygulaması

Sürüm 0.1.0-SNAPSHOT

5 Ağustos 2026

Kapsam notu

Bu belge mevcut proje kodu ve OpenAPI sözleşmesine göre hazırlanmıştır. Gerçek ESHS/TSA uçları, üretici sürücüleri, bağımsız birlikte çalışabilirlik ve hukuk/bilgi güvenliği kabulü tamamlanmadan mevzuata tam uygunluk iddiası oluşturmaz.

İçindekiler

- 1. Kılavuzun amacı ve hedef kitle
- 2. Platformu tanıyın
- 3. İmza formatları ve seviyeler
- 4. Hızlı başlangıç ve demo uygulamaları
- 5. Ortak API kuralları
- 6. Server-side entegrasyon
- 7. Client-side entegrasyon
- 8. İmza ve sertifika doğrulama
- 9. Güven deposu, TSA ve politikalar
- 10. Hata yönetimi ve güvenlik
- 11. Canlı ortam kontrol listesi
- 12. Doğrudan Java/JAR entegrasyonu
- 13. Çoklu imza: REST ve demo
- Ek A. Uç noktalar
- Ek B. Yardımcı kod parçaları

Okuma yolu

REST entegrasyonu için 4 ve 5. bölümlerden sonra server-side için 6, client-side için 7. bölüme geçin. REST kullanmadan JAR'ları doğrudan çağırıcaksanız 12; seri/paralel imza için 13. bölümü okuyun.

1. Kılavuzun amacı ve hedef kitle

Bu kılavuz E-İmza API platformunu çalıştıracak ekipler, kurumsal uygulamalarına imzalama ekleyecek geliştiriciler, akıllı kart/HSM yöneticileri ve doğrulama hizmetini kullanacak servis ekipleri için hazırlanmıştır.

- Platformun modüllerini ve güvenlik sınırlarını anlamak
- Web demo, yönetim ekranı, Smart Card Agent ve offline masaüstünü kullanmak
- Server-side kart veya HSM ile imza üretmek
- Client-side Smart Card Agent ile son kullanıcı kartından imza almak
- CAdES, XAdES ve PAdES imzalarını ve imzalayan sertifikayı doğrulamak
- Güvenilir kök/alt kök, RFC 3161 TSA ve doğrulama politikalarını yönetmek

Önemli

Özel anahtar hiçbir akışta karttan veya HSM'den dışarı çıkarılmaz. Sertifika public veridir; indirilen .cer dosyası özel anahtar veya PIN içermez.

2. Platformu tanıyın

Platform Java 21 ve Spring Boot tabanlı, çok modüllü bir elektronik imza altyapısıdır. İmzalama çekirdeği ile cihaz erişimi ayrılır; böylece aynı format üretim motoru server-side ve client-side akışlarda kullanılabilir.

Modül	Görev
signature-core	Ortak modeller, politika ve imzalama oturumu durum makinesi
certificate-validation	PKIX sertifika yolu, tarih, NES/politika, CRL ve OCSP kontrolleri
timestamp-client	RFC 3161 zaman damgası isteği ve token doğrulaması
signature-cades	CAdES B-B/B-T, paralel/seri imza, B-LT/B-LTA ve doğrulama
signature-xades	Detached/enveloped/enveloping, paralel/seri XAdES ve doğrulama
signature-pades	PDF ByteRange, ETSI.CAdES.detached ve incremental seri PAdES
smartcard-agent	Loopback PC/SC, ATR, PKCS#11, yerel PIN ve client-side kart imzası
signature-api	REST API, oturum orkestrasyonu, DB, yönetim, demo ve doğrulama
desktop-signing-demo	API/agent gerektirmeyen offline Swing CAdES uygulaması

2.1 Uygulama yüzeyleri

Adres / uygulama	Amaç
/demo/	Server-side veya client-side CAdES/XAdES/PAdES imzalama demosu
/multi-signature/	Mevcut imzalı dosyaya paralel veya seri imza demosu
/validation/	İmza doğrulama, imzalayan bilgileri ve .cer indirme
/admin/	Cihaz profili, güven deposu, TSA ve politika yönetimi
127.0.0.1:18443/agent/v1	Yalnız client-side akışta tarayıcı ile yerel agent arasındaki loopback API
desktop-signing-demo-...-exec.jar	Tamamen offline CAdES imzalama ve doğrulama

3. İmza formatları ve seviyeler

Format	Paketleme	Çıktı	Tipik kullanım
CAdES	DETACHED / ATTACHED	.p7s	Her tür dosyanın CMS tabanlı imzalanması
XAdES	DETACHED / ENVELOPED / ENVELOPING	.xades.xml	XML iş akışları ve XML içinde imza
PAdES	ENVELOPED	.signed.pdf	PDF'nin kendi içinde imza

Seviye	Anlam	Gereksinim
B_B	Temel imza	İmzalayan sertifikası ve kriptografik imza
B_T	İmza zaman damgalı	Etkin RFC 3161 TSA ve güvenilir TSA zinciri
B_LT	Uzun dönem kanıtları gömülü	Tam zincir ve CRL/OCSP kanıtları; CAdES yükseltme
B_LTA	Arşiv zaman damgalı	B-LT materyali ve arşiv TSA; CAdES yükseltme/yenileme

İlk test önerisi

TSA yapılandırılmadan B_B ile başlayın. B_T seçimi için TSA endpoint'i ve TSA güven kökü hazır olmalıdır.

4. Hızlı başlangıç ve demo uygulamaları

4.1 Gereksinimler ve paketleme

- Java 21
- Maven 3.8.8 veya üzeri
- Local test için H2; üretim için PostgreSQL
- Kart senaryoları için üreticinin onaylı PKCS#11 middleware'i

PowerShell / CMD uyarlaması

```
cd D:\ErbayProject
mvn "-Dmaven.repo.local=D:\ErbayProject\.m2\repository" -pl signature-api,smartcard-agent,desktop-signing-demo
-am -DskipTests package
```

Merkez API

```
java -jar .\signature-api\target\signature-api-0.1.0-SNAPSHOT-exec.jar --spring.profiles.active=local
```

Sağlık kontrolü: <http://localhost:8080/actuator/health>

4.2 Yönetim ekranı

<http://localhost:8080/admin/> adresi server key profillerini, güvenilir sertifikaları, TSA profilini ve doğrulama politikasını yönetir. Local örnek tenant UUID'si 11111111-1111-1111-1111-111111111111 değeridir.

1. Akıllı kart için PC/SC taramasıyla ATR'yi tespit edin veya manuel profil girin.
2. SMART_CARD için ATR, maske ve mutlak PKCS#11 yolu tanımlayın; slot otomatik bulunur.
3. HSM için ATR girmeyin; kütüphane yolu, slot ve credentialRef tanımlayın.
4. Kart sertifikasının kök ve alt kök CA sertifikalarını güven deposuna ekleyin.
5. B-T ve üzeri için TSA profilini ve TSA güven zincirini tanımlayın.
6. Ortam gereksinimine göre STRICT, CUSTOM veya AUDIT_ONLY politika modunu seçin.

4.3 Web imzalama ve doğrulama demoları

- İmzalama: <http://localhost:8080/demo/>
- Seri/paralel çoklu imza: <http://localhost:8080/multi-signature/>
- Doğrulama: <http://localhost:8080/validation/>
- Doğrulama sonucu imzalayanın subject/issuer, seri, geçerlilik, algoritma ve parmak izi bilgilerini gösterir.
- İmzalayanın public X.509 sertifikası DER .cer olarak indirilebilir.

4.4 Offline masaüstü

PowerShell

```
java -jar .\desktop-signing-demo\target\desktop-signing-demo-0.1.0-SNAPSHOT-exec.jar
```

Masaüstü uygulaması merkez API'ye bağlanmaz, HTTP sunucusu ve Smart Card Agent başlatmaz. Kart adı, ATR ve PKCS#11 kütüphane yolu yerel profile saklanır; PIN saklanmaz. Uygulama attached/detached CAdES imzalar, doğrular ve imzalayan sertifikasını .cer olarak kaydeder.

5. Ortak API kuralları

5.1 Temel adres ve başlıklar

Öge	Kural
Base URL	https://sunucu.example/api/v1 (local: http://localhost:8080/api/v1)
Authorization	Üretimde OAuth2 Bearer JWT; scope: eimza.sign, eimza.validate veya eimza.admin
X-Tenant-Id	Zorunlu UUID; üretimde doğrulanmış token tenant claim'i ile eşleşmelidir
Idempotency-Key	Oturum oluştururken zorunlu; aynı iş isteğinin tekrarını güvenli yönetir
X-Correlation-Id	İsteğe bağlı; uçtan uca log/olay korelasyonu için önerilir
Content-Type	JSON isteklerinde application/json

5.2 Base64 ve Base64URL

- Belge, sertifika ve artifact alanları standart Base64 kullanır.

- documentDigest.value, ham kart signature ve deviceSignature padding'siz Base64URL kullanır.
- Karakter dönüşümü yapılmamalı; özet doğrudan dosyanın baytlarından hesaplanmalıdır.

Java 21

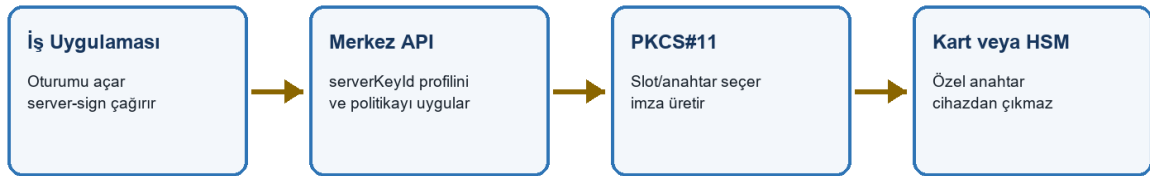
```
static String b64(byte[] value) {
    return Base64.getEncoder().encodeToString(value);
}
static String b64url(byte[] value) {
    return Base64.getUrlEncoder().withoutPadding().encodeToString(value);
}
```

5.3 Oturum yaşam döngüsü

Aşama	Client-side	Server-side
1	POST /signing-sessions	POST /signing-sessions
2	POST /{id}/manifest	POST /{id}/server-sign
3	agent-connected ve approve	API cihaz profilini çözer
4	Agent /signing-requests	PKCS#11 kart/HSM imzası
5	POST /{id}/complete	Artifact aynı çağrıda döner
6	GET /{id}/artifact	GET /{id}/artifact

6. Server-side entegrasyon

SERVER-SIDE AKIŞ



HSM PIN'i credentialRef ile secret ortamından alınır

Şekil 1 - Server-side imzalama güven sınırı

Server-side modda kart veya HSM, signature-api ile aynı güvenli sunucu ortamında erişilebilir durumdadır. İş uygulaması sürücü yolu veya slot göndermez; yalnız yönetici tarafından önceden oluşturulmuş serverKeyld kullanır.

6.1 Ne zaman seçilmeli?

- Kurumsal mühür veya servis imzası merkezi bir HSM'de tutuluyorsa
- İmza işlemi kullanıcının bilgisayarından bağımsız yürümeliyse
- Sunucuya fiziksel akıllı kart ve okuyucu bağlanmışsa
- Yüksek hacim için HSM partition/slot yönetimi gerekiyorsa

6.2 Ön hazırlık

Cihaz	Profil kuralları	PIN yönetimi
SMART_CARD	ATR + maske + mutlak PKCS#11 yolu; autoDiscoverSlot=true	Tek imza isteğinde pin alanı; DB/log'a yazılmaz
HSM	Mutlak PKCS#11 yolu + sabit slot; ATR yok	HTTP'de PIN yok; credentialRef ile ortam secret'ından

6.3 Adım 1 - belge özeti ve oturum

Java - istek gövdesi

```
byte[] document = Files.readAllBytes(Path.of("sozlesme.pdf"));
byte[] digest = MessageDigest.getInstance("SHA-256").digest(document);

String requestJson = ""
{
  "documentId": "%s",
  "documentDigest": {"algorithm": "SHA-256", "value": "%s"},
  "documentName": "sozlesme.pdf",
  "mediaType": "application/pdf",
  "size": %d,
  "format": "PADES",
  "targetLevel": "B_B",
  "turkishProfile": "P1",
  "purpose": "Sözleşme onayı",
  "signingMode": "SERVER_SIDE",
  "deviceId": null,
  "serverKeyId": "%s",
  "documentBase64": "%s",
  "signaturePackaging": "ENVELOPED",
  "signatureAlgorithm": "RSA_PKCS1_SHA256"
}
"".formatted(UUID.randomUUID(), b64url(digest), document.length,
serverKeyId, b64(document));
```

Java 21 HttpClient

```
HttpRequest create = HttpRequest.newBuilder(base.resolve("/api/v1/signing-sessions"))
  .header("Authorization", "Bearer " + accessToken)
  .header("X-Tenant-Id", tenantId)
  .header("Idempotency-Key", UUID.randomUUID().toString())
  .header("X-Correlation-Id", UUID.randomUUID().toString())
  .header("Content-Type", "application/json")
  .POST(HttpRequest.BodyPublishers.ofString(requestJson))
  .build();
HttpResponse<String> created = client.send(create,
  HttpResponse.BodyHandlers.ofString());
// 201 cevabındaki sessionId JSON kütüphanesiyle okunur.
```

6.4 Adım 2 - sunucuda imzala

HTTP - SMART_CARD

```
POST /api/v1/signing-sessions/{sessionId}/server-sign
Authorization: Bearer <token>
X-Tenant-Id: 11111111-1111-1111-1111-111111111111
Content-Type: application/json

{ "pin": "yalnız-smart-card-icin-tek-kullanimlik" }
```

HTTP - HSM

```
POST /api/v1/signing-sessions/{sessionId}/server-sign
...
{ "pin": null }
```

Başarılı cevap SignatureArtifact döndürür: artifactId, format, level, mediaType, sha256 ve artifactBase64. Base64 çözülerek .p7s, .xades.xml veya .signed.pdf yazılır.

Java

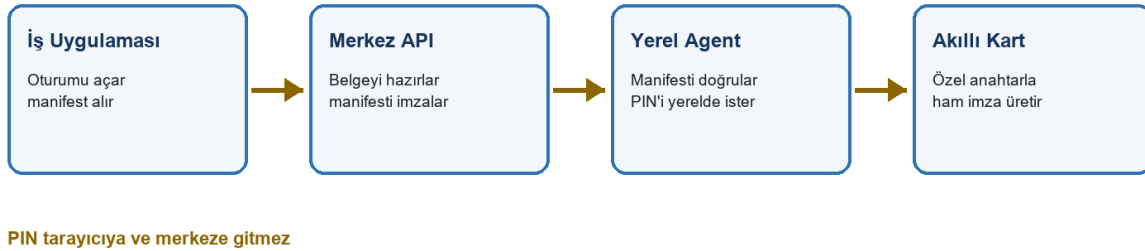
```
byte[] artifact = Base64.getDecoder().decode(artifactBase64);
Files.write(Path.of("sozlesme.signed.pdf"), artifact);
```

Server-side güvenlik

İş uygulaması PKCS#11 DLL yolu, HSM slotu veya credentialRef seçemez. Bu değerler eimza.admin yetkili yönetici tarafından önceden tanımlanır.

7. Client-side entegrasyon

CLIENT-SIDE AKIŞ



Şekil 2 - Client-side Smart Card Agent akışı

Client-side mod yalnız akıllı kart içindir; HSM desteklenmez. Tarayıcı WebCrypto ile belge özetini hesaplar, merkezden imzalı manifest alır ve loopback agent'a iletir. PIN yalnız agent'ın Swing penceresinde girilir.

7.1 Agent kurulumu ve yapılandırması

PowerShell

```

$env:EIMZA_AGENT_DEVICE_ID = "a8a0dc09-54ab-40b7-b404-bebd55ff1756"
$env:EIMZA_AGENT_MANIFEST_PUBLIC_KEY = (
  Invoke-RestMethod http://localhost:8080/api/v1/signing-configuration/manifest-key
).publicKey
$env:EIMZA_AGENT_ALLOWED_ORIGIN = "https://uygulama.example"
$env:SPRING_CONFIG_ADDITIONAL_LOCATION = "file:D:/eimza/agent.yml"

java -jar smartcard-agent-0.1.0-SNAPSHOT-exec.jar
  
```

Origin sınırı

EIMZA_AGENT_ALLOWED_ORIGIN yalnız gerçek iş uygulaması origin'ini içermelidir. Joker (*) veya rastgele web sitelerine izin verilmemelidir.

7.2 Cihaz kaydı

JavaScript

```

const agent = 'http://127.0.0.1:18443/agent/v1';
const identity = await fetch(agent + '/device').then(r => r.json());

await fetch('/api/v1/admin/client-devices', {
  method: 'POST',
  headers: {
    'Authorization': `Bearer ${token}`,
    'X-Tenant-Id': tenantId,
    'Content-Type': 'application/json'
  },
  body: JSON.stringify({
    deviceId: identity.deviceId,
    displayName: 'Muhasebe bilgisayarı',
    publicKey: identity.publicKey
  })
});
  
```

Üretimde cihaz kaydı normal son kullanıcı yetkisinden ayrılmalı; yönetici onayı, cihaz envanteri ve iptal süreci uygulanmalıdır. Ed25519 özel cihaz anahtarı agent'tan çıkmaz.

7.3 Kart ve public sertifika seçimi

JavaScript

```

const cards = await fetch(agent + '/cards').then(r => r.json());
const card = cards.find(item => item.status === 'READY');
if (!card) throw new Error('Hazır kart bulunamadı');

const certificates = await fetch(
  
```

```
agent + '/cards/' + encodeURIComponent(card.readerId) + '/certificates/refresh',
{ method: 'POST', headers: { 'X-Eİmza-Agent': '1' } }
).then(r => r.json());
const signer = certificates.find(item => item.hasPrivateKey());
```

Public sertifikalar mümkünse PIN'siz listelenir. Middleware public nesnelere otursuz erişim vermiyorsa üretici adaptörünün davranışına göre yerel PIN fallback'i gerekebilir. PIN hiçbir HTTP gövdesinde kabul edilmez.

7.4 Oturum, manifest ve kart imzası

JavaScript - oturum

```
const bytes = new Uint8Array(await file.arrayBuffer());
const digest = new Uint8Array(await crypto.subtle.digest('SHA-256', bytes));
const session = await apiJson('/signing-sessions', {
  method: 'POST',
  headers: { 'Idempotency-Key': crypto.randomUUID() },
  body: JSON.stringify({
    documentId: crypto.randomUUID(),
    documentDigest: { algorithm: 'SHA-256', value: b64url(digest) },
    documentName: file.name, mediaType: file.type, size: file.size,
    format: 'CADES', targetLevel: 'B_B', turkishProfile: 'P1',
    purpose: 'Kullanıcı onaylı imza', signingMode: 'CLIENT_SIDE',
    deviceId: identity.deviceId, serverKeyId: null,
    documentBase64: b64(bytes), signaturePackaging: 'DETACHED',
    signatureAlgorithm: 'RSA_PKCS1_SHA256'
  })
});
```

JavaScript - manifest ve tamamlama

```
const manifest = await apiJson('/signing-sessions/${session.sessionId}/manifest', {
  method: 'POST',
  body: JSON.stringify({
    readerId: card.readerId,
    certificateFingerprint: signer.fingerprintSha256,
    certificateBase64: signer.certificateBase64,
    signatureAlgorithm: 'RSA_PKCS1_SHA256'
  })
});

await apiJson('/signing-sessions/${session.sessionId}/agent-connected', {method:'POST'});
await apiJson('/signing-sessions/${session.sessionId}/approve', {method:'POST'});

const signed = await fetch(agent + '/signing-requests', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json', 'X-Eİmza-Agent': '1' },
  body: JSON.stringify({manifest: manifest.manifest, signature: manifest.signature})
}).then(r => r.json());

const artifact = await apiJson('/signing-sessions/${session.sessionId}/complete', {
  method: 'POST',
  body: JSON.stringify({
    signature: signed.signature,
    deviceSignature: signed.deviceSignature
  })
});
```

Kullanıcı onayı

Belge adı, boyutu, özet/amaç ve sertifika kullanıcıya PIN penceresinden önce gösterilmelidir. Beklenmeyen içerikte kullanıcı işlemi iptal etmelidir.

7.5 apiJson yardımcı fonksiyonu

JavaScript

```
async function apiJson(path, options = {}) {
  const headers = {
    'Authorization': `Bearer ${token}`,
    'X-Tenant-Id': tenantId,
    'X-Correlation-Id': crypto.randomUUID(),
    'Content-Type': 'application/json',
    ...(options.headers || {})
  };
  const response = await fetch('/api/v1' + path, {...options, headers});
  const body = await response.json();
  if (!response.ok) throw new Error(`${body.code}: ${body.detail || body.message}`);
  return body;
}
```

8. İmza ve sertifika doğrulama

8.1 İmza doğrulama isteği

HTTP

```
POST /api/v1/validations/signatures
Authorization: Bearer <token>
X-Tenant-Id: 11111111-1111-1111-1111-111111111111
Content-Type: application/json
```

```
{
  "format": "CADES",
  "signaturePackaging": "DETACHED",
  "content": "<original-belge-Base64>",
  "signature": "<p7s-Base64>",
  "validationTime": null
}
```

Format	signature	content
CAdES ATTACHED	DER CMS/CAdES Base64	null
CAdES DETACHED	DER CMS/CAdES Base64	Orijinal belge Base64
XAdES ENVELOPED/ ENVELOPING	İmzalı XML Base64	null
XAdES DETACHED	İmzalı XML Base64	Orijinal belge Base64
PAdES	İmzalı PDF Base64	null

8.2 Sonucu yorumlama

Ana sonuç	Yorum
VALID	Kriptografik, sertifika ve etkin zorunlu politikalar geçti
INVALID	Kriptografik/biçimsel hata veya kesin politika ihlali var; imza kabul edilmez
INDETERMINATE	Kanıt/hizmet eksik, politika pasif veya güven kararı tamamlanamadı; otomatik geçerli sayılmaz

checks dizisi her kontrolün code, indication, message, evidenceDigest ve details alanlarını taşır. Karar verirken yalnız summary metnine değil mainIndication ve checks alanlarına bakın.

8.3 İmzalayan bilgileri ve sertifika indirme

JavaScript

```
const signer = report.signer;
console.log(signer.commonName, signer.subjectDn, signer.issuerDn);
console.log(signer.validFrom, signer.validUntil, signer.sha256Fingerprint);

const der = Uint8Array.from(atob(signer.certificateBase64), c => c.charCodeAt(0));
const url = URL.createObjectURL(new Blob([der], {type: 'application/pkix-cert'}));
const link = document.createElement('a');
link.href = url;
link.download = `imzalayan-${signer.certificateSerialNumber}.cer`;
link.click();
URL.revokeObjectURL(url);
```

Sertifika public veridir

certificateBase64 yalnız DER X.509 public sertifikayı içerir. Bununla imza üretilemez; özel anahtar donanımda kalır.

8.4 Sertifika doğrulama

HTTP

```
POST /api/v1/validations/certificates
{
  "certificate": "<leaf-DER-Base64>",
  "intermediateCertificates": [ "<alt-kök-DER-Base64>" ],
  "validationTime": null
}
```

9. Güven deposu, TSA ve politikalar

9.1 Güvenilir kök ve alt kökler

- Yalnız CA sertifikaları ROOT veya INTERMEDIATE olarak eklenir; son kullanıcı sertifikası trust anchor yapılmaz.
- Ekleme/güncelleme/silme yeni bir tarihsel snapshot sürümü üretir; geçmiş kararlar izlenebilir kalır.
- Sertifika zinciri imza zamanı veya doğrulama zamanı için ilgili snapshot'a göre kurulmalıdır.

HTTP

```
POST /api/v1/admin/trusted-certificates
{
  "certificate": "<PEM-veya-DER-Base64>",
  "trustType": "ROOT",
  "displayName": "Kurumsal güvenilir kök",
  "enabled": true
}
```

9.2 RFC 3161 TSA

- TSA endpoint, timeout, sağlayıcı ID ve credentialRef yönetim ekranından sürümlenir.
- Gerçek Authorization değeri DB'ye değil credentialRef'in işaret ettiği secret ortamına yazılır.
- TSA sertifika zincirinin kökü de güven deposunda bulunmalıdır.
- B-T/B-LTA üretiminde timestamp policy OID ve token doğrulaması izlenmelidir.

9.3 Doğrulama politikaları

Politika	Amaç
QC compliance	Nitelikli sertifika/QC ifadelerini denetler
Certificate policy	Zorunlu sertifika politika OID'lerini denetler
Revocation	CRL/OCSP kanıtını ve erişilebilirliğini denetler
Signature policy	İmza politikası OID/özetini denetler
Signing certificate validity	Sertifikanın imzalama zamanındaki geçerliliğini denetler

Production kuralı

Sertifika tarih kontrolü production profilinde zorunlu aktiftir. Local/test ortamında süresi dolmuş kartla yalnız donanım entegrasyon testi için pasifleştirilirse sonuç hukuken geçerli imza sayılmaz.

10. Hata yönetimi ve güvenlik

10.1 Problem Details

Örnek hata

```
{
  "type": "https://errors.eimza.local/...",
  "title": "SERVER_PKCS11_SIGNING_FAILED",
  "status": 422,
}
```

```

"detail": "Sunucu PKCS#11 imzalama işlemi tamamlanamadı.",
"code": "SERVER_PKCS11_SIGNING_FAILED",
"correlationId": "...",
"retryable": false,
"timestamp": "..."
}

```

Durum	Uygulama davranışı
400	İstek şemasını ve Base64/Base64URL kullanımını düzeltin; otomatik tekrar etmeyin
401/403	Token, scope ve tenant claim eşleşmesini kontrol edin
404	Tenant kapsamındaki session/serverKey kaydını kontrol edin
409	Oturum durumunu GET ile okuyun; idempotency sonucunu kullanın
422	Kart/sertifika/politika/format ayrıntısını code ve checks üzerinden çözün
5xx / retryable=true	Sınırlı exponential backoff ve aynı Idempotency-Key ile tekrar değerlendirin

10.2 Zorunlu güvenlik kontrolleri

- TLS olmadan üretim trafiği taşımayın; loopback agent dışında HTTP kullanmayın.
- PIN, HSM secret, token ve özel anahtar materyalini loglamayın.
- Client-side PIN'i web formuna koymayın; yalnız agent Swing penceresi kullanmalıdır.
- Belge içeriğini ve Base64 artifact'i uygulama loglarına yazmayın.
- OAuth scope ve tenant izolasyonunu her uçta uygulayın.
- Idempotency-Key'i iş işlemi bazında üretin; rastgele tekrarlar yeni imza oluşturmasın.
- Manifest Ed25519 anahtarını üretimde KMS/HSM/secret manager ile sürümleyin.
- Agent origin allowlist, imzalı dağıtım ve cihaz iptal prosedürü kullanın.
- PKCS#11 sürücülerini yalnız üretici/kurum onaylı kaynaktan kurun.

11. Canlı ortam kontrol listesi

1. PostgreSQL, yedekleme ve Flyway migration yetkilerini doğrulayın.
2. OIDC issuer, audience, JWS algoritmaları ve tenant claim eşleşmesini doğrulayın.
3. Manifest Ed25519 anahtar çiftini kalıcı ve sürümlü secret olarak sağlayın.
4. Güvenilir kök/alt kök ve TSA zincirini kurum politikasıyla onaylatın.
5. OCSP/CRL/TSA ağ çıkışlarını allowlist ve timeout kurallarıyla açın.
6. HSM slot/partition ve credentialRef değerlerini iki kişi kontrolüyle tanımlayın.
7. Agent paketini kod imzalı dağıtın; origin ve cihaz kayıt yaşam döngüsünü yönetin.
8. CAdES/XAdES/PAdES için pozitif, bozuk içerik, yanlış paketleme ve süresi dolmuş sertifika testlerini çalıştırın.
9. Yük, felaket kurtarma, bağımsız sızma ve fiziksel kart kabul testlerini tamamlayın.
10. Correlation ID, metrik, alarm ve olay müdahale runbook'larını operasyon ekibine teslim edin.

12. Doğrudan Java/JAR entegrasyonu

Bu bölüm REST uçlarını veya demo ekranlarını kullanmadan signature-* JAR modüllerini kendi Java 21 uygulamanızda çağırmaı gösterir. Örnekler proje ile birlikte sürümlenen JAVA_KUTUPHANE_ENTTEGRASYONU.md kaynağından üretilir.

Temel sözleşme

prepare ile imzalanacak baytları üretin; kart/HSM/PrivateKey ile imzalayın; completeBaseline veya completeWithTimestamp ile standart artifact'i tamamlayın. Özel anahtar format modüllerine verilmez.

Bu rehber REST API veya demo web sayfası kullanmadan `signature-\*` JAR modüllerini doğrudan bir Java 21 uygulamasına eklemeyi gösterir. Temel desen her formatta aynıdır:

1. `prepare(...)` ile imzalanacak yapı ve özet hazırlanır.
2. Uygulama kendi `PrivateKey`, PKCS#11 kart veya HSM adaptörüyle imza üretir.
3. `completeBaseline(...)` veya `completeWithTimestamp(...)` artifact'i üretir.

Maven bağımlılıkları

xml

```
<dependency>
  <groupId>tr.gov.eimza</groupId>
  <artifactId>signature-cades</artifactId>
  <version>0.1.0-SNAPSHOT</version>
</dependency>
<dependency>
  <groupId>tr.gov.eimza</groupId>
  <artifactId>signature-xades</artifactId>
  <version>0.1.0-SNAPSHOT</version>
</dependency>
<dependency>
  <groupId>tr.gov.eimza</groupId>
  <artifactId>signature-pades</artifactId>
  <version>0.1.0-SNAPSHOT</version>
</dependency>
```

Akıllı kart profil/ATR/PKCS#11 yardımcılarını da kullanacak uygulama ayrıca:

xml

```
<dependency>
  <groupId>tr.gov.eimza</groupId>
  <artifactId>smartcard-agent</artifactId>
  <version>0.1.0-SNAPSHOT</version>
</dependency>
```

Geliştirme sürümünü yerel Maven deposuna kurmak için:

powershell

```
mvn "-Dmaven.repo.local=D:\ErbayProject\.m2\repository" install
```

Yazılım anahtarıyla örnek imzalayıcı

Bu yardımcı test ve sunucu `PrivateKey` örnekleri içindir. Akıllı kart/HSM entegrasyonunda özel anahtar yerine cihazın `sign` çağırısı kullanılmalıdır.

java

```
static byte[] signBytes(
    byte[] bytes, PrivateKey privateKey, String jcaAlgorithm) throws Exception {
    Signature signer = Signature.getInstance(jcaAlgorithm);
    signer.initSign(privateKey);
    signer.update(bytes);
    return signer.sign();
}
```

CAdES-B-B doğrudan kullanım

java

```
byte[] document = Files.readAllBytes(Path.of("sozlesme.xml"));
X509Certificate certificate = loadCertificate();
PrivateKey privateKey = loadPrivateKeyForTest();
```

```

CadesSignatureService service = new CadesSignatureService();
CadesSigningPreparation preparation = service.prepare(
    document,
    certificate,
    CadesSignatureAlgorithm.RSA_PKCS1_SHA256,
    null, // Kurumsal SignaturePolicy gerekirse verilir
    Instant.now(),
    false, // false=DETACHED, true=ATTACHED
    true); // sertifika tarih kontrolü

byte[] rawSignature = signBytes(
    preparation.signedAttributes(),
    privateKey,
    preparation.signatureAlgorithm().jcaName());

CadesSignatureResult result =
    service.completeBaseline(preparation, rawSignature);
Files.write(Path.of("sozlesme.p7s"), result.encodedSignature());

```

Doğrulama:

java

```

CadesVerificationResult verification =
    new CadesSignatureVerifier().verifyDetached(
        document, result.encodedSignature(), null, Set.of());
if (!verification.cryptographicValidity()) {
    throw new IllegalStateException("CAdES geçersiziz");
}

```

XAdES-B-B doğrudan kullanım

java

```

byte[] document = Files.readAllBytes(Path.of("belge.xml"));
byte[] digest = MessageDigest.getInstance("SHA-256").digest(document);
XadesSignatureService service = new XadesSignatureService();

XadesSigningPreparation preparation = service.prepare(
    SignaturePackaging.DETACHED,
    "urn:uuid:" + UUID.randomUUID(),
    document,
    "application/xml",
    digest,
    certificate,
    "RSA_PKCS1_SHA256",
    Instant.now(),
    true);

byte[] rawSignature =
    signBytes(preparation.signedInfo(), privateKey, "SHA256withRSA");
XadesSignatureResult result =
    service.completeBaseline(preparation, rawSignature);
Files.write(Path.of("belge.xades.xml"), result.encodedSignature());

```

PAdES-B-B doğrudan kullanım

java

```

byte[] pdf = Files.readAllBytes(Path.of("sozlesme.pdf"));
PadesSignatureService service = new PadesSignatureService();

PadesSigningPreparation preparation = service.prepare(
    pdf,
    certificate,
    CadesSignatureAlgorithm.RSA_PKCS1_SHA256,
    null,
    Instant.now(),
    certificate.getSubjectX500Principal().getName(),
    "Sözleşme onayı",
    true);

byte[] rawSignature = signBytes(
    preparation.cmsPreparation().signedAttributes(),
    privateKey,
    preparation.cmsPreparation().signatureAlgorithm().jcaName());
PadesSignatureResult result =
    service.completeBaseline(preparation, rawSignature);
Files.write(Path.of("sozlesme.signed.pdf"), result.encodedPdf());

```

Akıllı kart ile doğrudan imza

`Pkcs11TokenService` Spring uygulamasına constructor injection ile alınabilir. Public sertifikalar PIN'siz listelenir; PIN yalnız `sign` çağrısında kullanılır.

java

```
@Service
final class CardCadesSigner {
    private final Pkcs11TokenService token;
    private final CadesSignatureService cades = new CadesSignatureService();

    CardCadesSigner(Pkcs11TokenService token) {
        this.token = token;
    }

    byte[] sign(
        byte[] document,
        CardProfile profile,
        CardCertificate selected,
        X509Certificate certificate,
        char[] pin) {
        CadesSignatureAlgorithm algorithm =
            CadesSignatureAlgorithm.RSA_PKCS1_SHA256;
        CadesSigningPreparation preparation = cades.prepare(
            document, certificate, algorithm, null,
            Instant.now(), false, true);
        byte[] raw = null;
        try {
            raw = token.sign(
                profile,
                selected.fingerprintSha256(),
                preparation.digestToSign(),
                algorithm.name(),
                pin);
            return cades.completeBaseline(preparation, raw).encodedSignature();
        } finally {
            Arrays.fill(pin, '\0');
            if (raw != null) Arrays.fill(raw, (byte) 0);
        }
    }
}
```

CAdES paralel ve seri imza

java

```
// Önceki .p7s içine bağımsız ikinci SignerInfo ekle
CadesSigningPreparation coSign = cades.prepareParallel(
    previousP7s, originalDocument, secondCertificate,
    CadesSignatureAlgorithm.RSA_PKCS1_SHA256,
    null, Instant.now(), false, true);
byte[] coRaw = signBytes(
    coSign.signedAttributes(), secondPrivateKey, "SHA256withRSA");
byte[] parallelP7s =
    cades.completeBaseline(coSign, coRaw).encodedSignature();

// Üst seviye 0 numaralı imzanın CMS counter-signature'ını üret
CadesSigningPreparation counter = cades.prepareCounterSignature(
    parallelP7s, 0, approverCertificate,
    CadesSignatureAlgorithm.RSA_PKCS1_SHA256,
    null, Instant.now(), true);
byte[] counterRaw = signBytes(
    counter.signedAttributes(), approverPrivateKey, "SHA256withRSA");
byte[] serialP7s =
    cades.completeBaseline(counter, counterRaw).encodedSignature();
```

XAdES paralel ve seri imza

java

```
XadesSigningPreparation coSign = xades.prepareParallel(
    previousXades,
    SignaturePackaging.DETACHED,
    documentUri,
    originalDocument,
    "application/xml",
    documentDigest,
    secondCertificate,
    "RSA_PKCS1_SHA256",
    Instant.now(),
```



```

        true);
byte[] coRaw = signBytes(
    coSign.signedInfo(), secondPrivateKey, "SHA256withRSA");
byte[] parallelXades =
    xades.completeBaseline(coSign, coRaw).encodedSignature();

XadesSigningPreparation counter = xades.prepareCounterSignature(
    parallelXades, 0, approverCertificate,
    "RSA_PKCS1_SHA256", Instant.now(), true);
byte[] counterRaw = signBytes(
    counter.signedInfo(), approverPrivateKey, "SHA256withRSA");
byte[] serialXades =
    xades.completeBaseline(counter, counterRaw).encodedSignature();

```

Paralel XAdES için `DETACHED` veya `ENVELOPING` kullanılır. `ENVELOPED + PARALLEL` bilinçli olarak reddedilir.

PAdES seri imza

java

```

PadesSigningPreparation secondApproval = pades.prepareSequential(
    previouslySignedPdf,
    secondCertificate,
    CadesSignatureAlgorithm.RSA_PKCS1_SHA256,
    null,
    Instant.now(),
    "İkinci Onaycı",
    "İkinci aşama onayı",
    true);
byte[] raw = signBytes(
    secondApproval.cmsPreparation().signedAttributes(),
    secondPrivateKey,
    "SHA256withRSA");
byte[] signedTwice =
    pades.completeBaseline(secondApproval, raw).encodedPdf();

```

Üretim notları

- Test dışındaki private key'leri dosyadan yüklemeyin; kart/HSM/KMS adaptörü kullanın.
- PIN, private key, raw imza girdisi ve belge Base64 değerini loglamayın.
- B-T için `TimestampClient` vererek `completeWithTimestamp` çağırın.
- Sertifika zinciri ve politika doğrulamasını artifact üretiminden ayrı uygulayın.
- Aynı iş isteğini uygulama katmanında idempotent yönetin.

13. Çoklu imza: REST ve demo

Format	PARALLEL	SERIAL	Paketleme notu
CAdES	Yeni üst seviye SignerInfo	Hedef imzada counterSignature	ATTACHED / DETACHED
XAdES	Bağımsız ds:Signature	xades:CounterSignature	Paralel: DETACHED / ENVELOPING
PAdES	Desteklenmez	Incremental PDF revision	ENVELOPED

POST /api/v1/signing-sessions

```

{
  "...": "normal oturum alanları",
  "format": "CADES",
  "multiSignatureType": "PARALLEL",
  "existingArtifactBase64": "<önceki-p7s-Base64>",
  "targetSignatureIndex": 0,
  "documentBase64": "<orijinal-belge-Base64>"
}

```

- SINGLE varsayılandır; eski istemciler yeni alanları göndermeden çalışır.

- PARALLEL ve SERIAL için existingArtifactBase64 zorunludur.
- CAdES/XAdES PARALLEL için orijinal belge gerekir; PAdES yalnız SERIAL kabul eder.
- XAdES ENVELOPED + PARALLEL reddedilir; DETACHED veya ENVELOPING seçilir.
- targetSignatureIndex sıfır tabanlıdır ve CAdES/XAdES seri imzada hedef imzayı seçer.
- İki Base64 alanı için varsayılan gövde sınırı 70 MiB'dir; EIMZA_MAXIMUM_REQUEST_BYTES ile ortam kapasitesine göre değiştirilebilir.

13.1 Demo adımları

1. İlk imzayı /demo/ sayfasında üretip artifact'i indirin.
2. /multi-signature/ sayfasında mevcut imzalı artifact'i seçin.
3. CAdES/XAdES PARALLEL için orijinal dosyayı da seçin; SERIAL için hedef indeksi girin.
4. Client-side kart/sertifika veya server-side serverKeyId profilini seçerek imzalayın.
5. Yeni artifact'i indirin ve /validation/ sayfasında doğrulayın.

Doğrulama kapsamı

CAdES'teki tüm üst seviye ve iç içe SignerInfo değerleri, XAdES'teki tüm ds:Signature/CounterSignature öğeleri ve PAdES'teki tüm imza sözlükleri doğrulanır.

Ek A. Uç noktalar

Yöntem ve yol	Amaç
POST /signing-sessions	İmzalama oturumu oluşturur
GET /signing-sessions/{id}	Oturum durumunu getirir
POST /signing-sessions/{id}/manifest	Client-side imzalı manifest üretir
POST /signing-sessions/{id}/agent-connected	Agent bağlantısını kaydeder
POST /signing-sessions/{id}/approve	Kullanıcı onayı sonrası kart imzasına geçer
POST /signing-sessions/{id}/complete	Client-side ham imzayı tamamlar
POST /signing-sessions/{id}/server-sign	Server-side kart/HSM imzası üretir
GET /signing-sessions/{id}/artifact	Tamamlanmış imza çıktısını getirir
POST /validations/signatures	CAdES/XAdES/PAdES doğrular
POST /validations/certificates	X.509 sertifika ve zincirini doğrular
POST /signatures/cades/augmentations	CAdES B-LT/B-LTA yükseltir/yeniler
/admin/trusted-certificates	Güven deposu CRUD ve sürümleri
/admin/server-key-profiles	Akıllı kart/HSM profil yönetimi
/admin/tsa-profile	Sürümlü TSA profili
/admin/validation-policy	Sürümlü doğrulama politikası

Ek B. Yardımcı kod parçaları

Java 21

```
static void requireSuccess(HttpResponse<String> response) {
    if (response.statusCode() < 200 || response.statusCode() >= 300) {
        throw new IllegalStateException(
            "E-imza API hatası: HTTP " + response.statusCode() + " - " + response.body());
    }
}

static String sha256Hex(byte[] value) throws Exception {
    return HexFormat.of().formatHex(
        MessageDigest.getInstance("SHA-256").digest(value));
}
```

Tarayıcı JavaScript

```
const b64 = bytes => {
    let text = '';
    for (let i = 0; i < bytes.length; i += 32768)
        text += String.fromCharCode(...bytes.slice(i, i + 32768));
    return btoa(text);
};
const b64url = bytes => b64(bytes)
    .replaceAll('+', '-') .replaceAll('/', '_').replaceAll('=', '');
```

Belge ve proje kaynakları

- OpenAPI: signature-api/src/main/resources/static/openapi/e-signature-api-v1.yaml
- Agent OpenAPI: smartcard-agent/src/main/resources/static/openapi/smartcard-agent-v1.yaml
- Yönetim/agent rehberi: YONETIM_EKRANI_VE_AGENT_REHBERI.md
- Offline masaüstü: desktop-signing-demo/README.md
- Standart ve faz kararları: E_IMZA_PROJE_PLANI.md, FAZ_10_UCTAN_UCA_CADES_XADES_PADES.md ve FAZ_11_COKLU_IMZA_VE_JAVA_KUTUPHANE_ENTTEGRASYONU.md
- Doğrudan JAR örnekleri: JAVA_KUTUPHANE_ENTTEGRASYONU.md